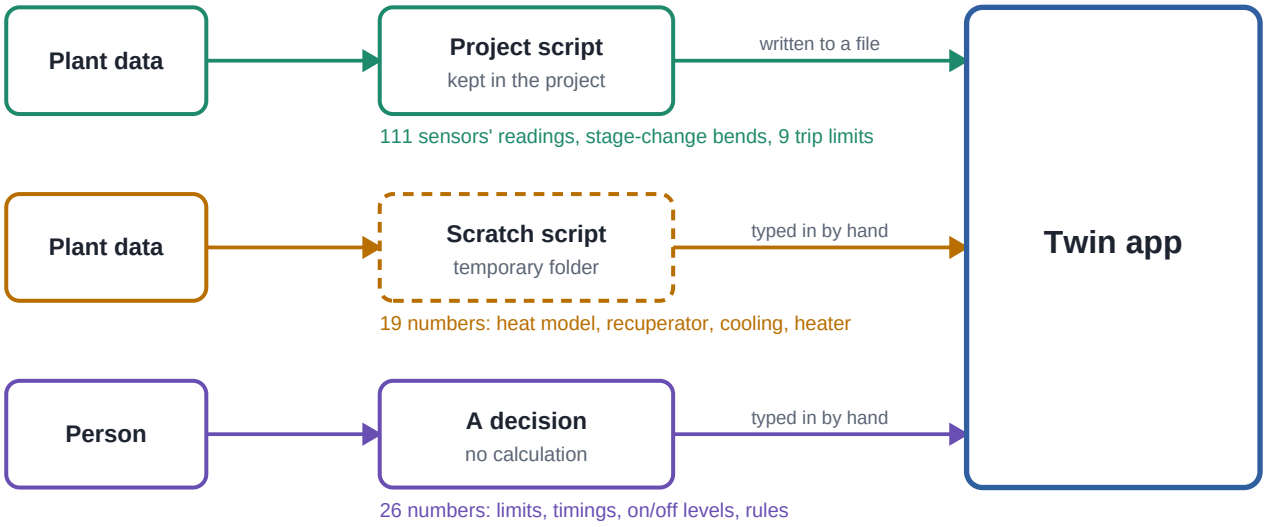
# Does the twin use numbers that no notebook ever calculated?

**Yes.** No number in the twin app comes straight from a Jupyter notebook. About 45 numbers were typed into the app by hand: 19 were calculated, but in temporary files, and 26 were simply chosen.

Think of the twin app as a recipe card. Every number in it is an ingredient amount. We asked one question for each amount: *can someone else see where it came from?*




Jupyter notebooks: no direct route into the app

Green: a project script calculates the numbers and writes them to a file. Amber: calculated, but the working was left in a temporary folder. Violet: decided by a person.

| Route | What it means | Where to read more |
|---|---|---|
| Green · kept | A project script calculates them from plant data, and anyone can run it again. | Page 4 |
| Amber · 19 numbers | Calculated from plant data, but in scratch files that Windows may delete. | Page 2 |
| Violet · 26 numbers | Sensible picks, rules and safety margins that nobody calculated. | Page 3 |

## Why it matters

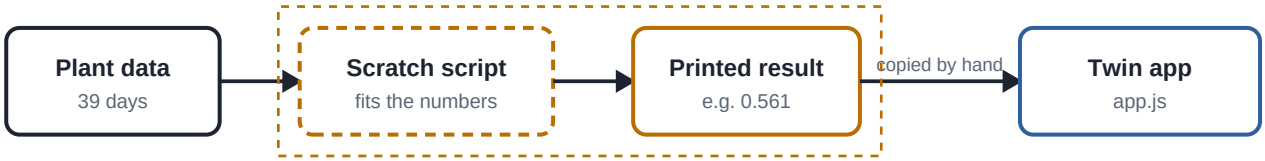
When a reviewer asks "why 0.561?", we should be able to point to one place and re-run it. Right now that is possible only for the green route. For the amber route, the only record could disappear. For the violet route, nothing records that the number is an assumption and not a measurement.

**If you remember one thing:** The twin's green numbers are fine. The amber ones need their working moved into a notebook, and the violet ones need to be labelled as assumptions.

AMBER · GROUP 1 OF 3

# Measured, but the notes were not kept

These 19 numbers are real results from plant data. The problem is where the working sits: in short scripts in Claude's temporary folder, not in a notebook in the project.




**Temporary folder: Windows can empty it at any time**

If it goes, nobody can show how these numbers were found

"Fitting" means trying many values until the twin's curve lies as close as possible to the plant's real curve. It is a calculation, so it can be repeated, but only if the script is still there.

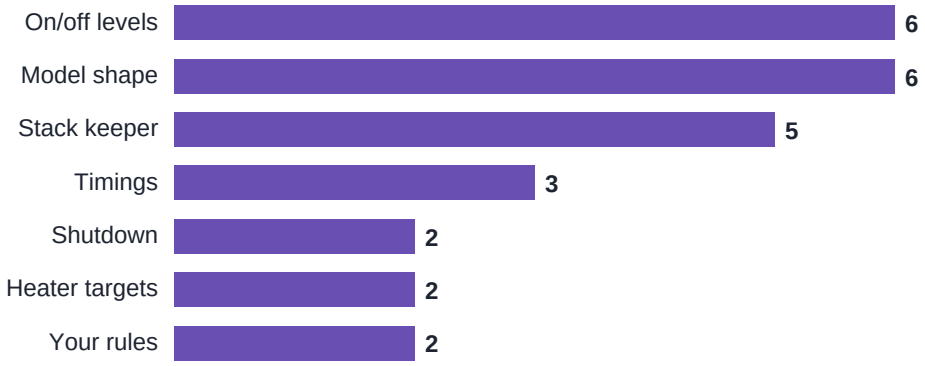
| What it is | What it does in the twin | Numbers | Scratch file · date |
|---|---|---|---|
| Stack heat model | How fast the stack warms or cools from the air, the heaters, the gas and the electric current | 0.561, 0.316, 7.62, 0.386, 6.34, 0.0121 | fix/fit1–5.py · 3 Nov |
| Air-heater lift | How much the 40 kW air heater can warm the incoming air | 268 | fix/fitair.py · 3 Nov |
| Recuperator | How much exhaust heat goes back into the incoming air, and how slowly it follows the stack | 0.8515, 0.872, 32.4, 0.852, 31.8 | fix/fitrec.py · 4 Nov |
| Cooling after hot standby | How fast the stack cools once the heaters go off | 0.0605, 1.65, 4.2 | hsc/fit–fit5.py · 4 Nov |
| Heater efficiency | Share of the air heater's electricity that ends up in the air | 0.89 | eaph/a1.py · 4 Nov |
| Air-heater targets | Plant's typical air temperature in fuel switch and part load | 580 °C, 600 °C | hot/part45.py |

**If you remember one thing:** These numbers are trustworthy, but their proof sits in a folder that can be emptied. Copy it into a project notebook.

VIOLET · GROUP 2 OF 3

# Chosen, not measured

Every model needs some numbers that nobody calculates. They are like a recipe's "a pinch of salt": a sensible choice, a safety margin or a rule. That is normal. The gap is that the app does not mark them as choices.




The 26 chosen numbers, grouped by kind. Source: a read-through of the twin's code (app.js) on 5 Nov 2026.

| Kind | Count | In plain words |
|---|---|---|
| On/off levels | 6 | When does the twin count gas, nitrogen, hydrogen, current or air as "on"? Plus the 25 °C room temperature. |
| Model shape | 6 | Shape choices in the heat model: air effect grows as air^1.5, a 10-minute air lag, and the 300 to 250 °C hand-over zone. |
| Stack keeper | 5 | The twin's own safety thermostat: start acting at 660 °C, add 200 ln/min of air per minute, then lower the heater by 5 °C per minute (not below 500 °C). |
| Timings | 3 | Purge 16 min, unloading 10 min, fuel switch 10 min. Typical plant values, but no notebook calculates them. |
| Shutdown | 2 | Cool with 9000 ln/min of air and switch the air off below 40 °C. Read off a single plant trip (#8). |
| Heater targets | 2 | Air heater at 300 °C for a cold start and 500 °C for hot standby. |
| Your rules | 2 | Purge only from hot standby or part load. Part load always goes back to hot standby. Decided with you on 4 Nov. |

The stack keeper exists only in the twin. The plant never ran part load for longer than 44 minutes, so the data cannot show how the plant would hold the stack temperature over a longer run.

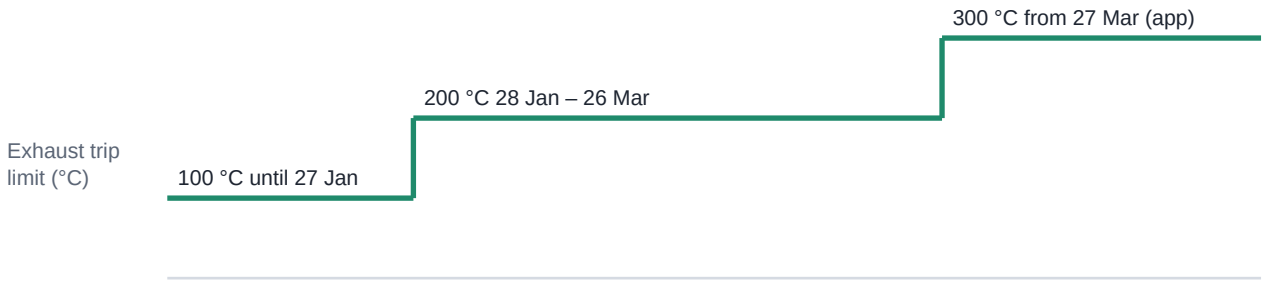
**If you remember one thing:** Chosen numbers are fine, as long as they are written down as assumptions so nobody mistakes them for plant measurements.

GREEN · GROUP 3 OF 3

# Measured and kept

This is the safe route. Scripts stored in the project read the plant data and write files that the app loads. Anyone can run them again and get the same numbers. They are Python scripts, not notebooks, but they leave a clear trail.

| Script in the project | What it gives the app |
|---|---|
| 05_twin_2d/tools/prep.py | How each of the 111 sensors reads in each stage, plus how it glides from one stage to the next |
| 05_twin_2d/tools/bends.py | How sharply the stack temperature turns at each stage change. It uses the same code as the slopes notebook. |
| 03_data_analysis/scripts/trip_check.py | The 9 temperature trip limits, taken from the plant's sensor map document |




The plant raised its exhaust trip limit twice. The app uses today's 300 °C.

**A correction to my earlier chat answer.** I said the app's 300 °C exhaust limit looked wrong, because most trips happened at just over 200 °C. That was a mistake. The plant raised the limit from 200 to 300 °C on 27 March, after those trips. The app uses the current value, which is right.

**If you remember one thing:** Green numbers can be rebuilt at any time by re-running a project script. This is the standard the other two groups should reach.

NEXT STEPS

# What to do next

The fix is small. We don't change the twin's behaviour. We only move the working to a place where it is kept and visible.

| Step | What happens |
|---|---|
| 1. Make one notebook | Copy the amber fit scripts into 03_data_analysis/twin_fits.ipynb |
| 2. Re-run and compare | Check that each printed number still matches the app |
| 3. List the choices | Add an "assumptions" table for the 26 violet numbers |
| 4. Stop hand-typing (optional) | The notebook writes a file the app loads, as prep.py does |

## Decisions for you

| | Question | My suggestion |
|---|---|---|
| A | Move the 19 amber fits into one notebook? | Yes. This is the only route with a real risk of being lost. |
| B | Write down the 26 violet numbers as an assumptions table? | Yes, in the same notebook and in the twin's README. |
| C | If a re-run gives a slightly different number, which one wins? | The notebook's. Update the app and note the change. |
| D | Let the notebook write the numbers to a file the app loads (step 4)? | Later. Do it once the numbers stop changing. |

Not checked yet: I have not re-run the scratch scripts, so I don't know whether every printed number still matches the app exactly. Step 2 does that check.

**If you remember one thing:** One notebook that calculates every amber number and lists every violet choice closes the gap.